Quality, contest calendar and project documentation

Suryadev Singh

This guide connects test strategy with the contest calendar's caching and failure behavior, and explains how documentation helps someone verify what is implemented. Avoid treating tests or fixtures as proof of live production behavior.

Contribution reminder: This is a repository-aligned role guide. Describe work you personally performed; confirm individual contribution details with your team.

Explain the system in one flow

Contest provider	**THEN**	**Calendar service + cache**	**THEN**	**Controller/API**	**THEN**	**Frontend contest view**	**THEN**	**Tests + documented limits**

Core ideas to explain

Contest cache

The calendar service caches provider results for a configurable TTL (default 10 minutes) and uses a single in-flight refresh to avoid duplicate concurrent fetches.

Resilience

After a provider failure, the service uses a retry delay and can retain stale data. Status distinguishes stale or unavailable data from a successful current fetch.

Fixtures

The fixture provider is a deterministic local test/demo option selected by configuration; it does not fetch actual upstream contests.

Quality checks

Backend tests cover security, adapters, controllers, persistence and contest behavior. Frontend tests cover API handling, themes, accessibility markup and calendar behavior.

Documentation

Architecture, integration, contribution and validation docs communicate current behavior, known limits and how to reproduce checks.

Viva quick preparation

Use these as short answer outlines. Explain the idea in your own words and point to the relevant repository code.

Why cache contests?

Contest schedules change less often than the UI is read. A TTL reduces repeated upstream calls and helps keep the screen responsive.

How does it avoid duplicate refresh calls?

Concurrent requests can share one in-flight future so they wait for the same provider refresh rather than launching redundant fetches.

What happens when the provider fails?

The service applies a retry delay and may return the previous response with a stale status; if no usable data exists, it reports unavailable.

Are fixture contests real?

No. They are deterministic illustrative data for local testing/demo and are used only when the fixture provider is explicitly configured.

What do tests prove?

They check the specified behavior under test conditions. They do not prove external provider uptime, production credentials, or every real deployment condition.

Where to point in the repository

```
01 backend/src/main/java/com/kodbtw/service/ContestCalendarService.java
02 backend/src/main/java/com/kodbtw/contest/
03 backend/src/test/java/
04 frontend/tests/
05 documentation/TESTING_AND_VALIDATION.md
06 documentation/PROJECT_AUDIT.md
07 documentation/TEAM_CONTRIBUTIONS.md
```

30-second presentation

“My focus is quality and the contest calendar. The calendar caches upstream results for a configurable period and shares an in-flight refresh when requests overlap. If the source fails, it can serve clearly marked stale data or report unavailable. Tests cover these behaviors and the frontend states; fixture contests are local illustrative data, not real schedules. The docs record the tested behavior and the limits.”

Be precise Do not describe fixture data as live events. A green test suite does not guarantee an external integration or production deployment. Confirm exact personal contribution history with the team before presenting it as individual authorship.